

Chapter 2: Derivatives and Gradients

Algorithms for Optimization, 2nd ed. (2026)

Kochenderfer & Wheeler
Slides prepared for lecture

MIT Press

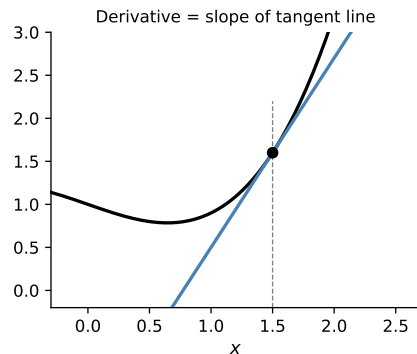
2026

Chapter Outline

1. Scalar Derivatives
2. Derivatives in Multiple Dimensions
3. Numerical Differentiation
4. Automatic Differentiation
5. Regression Gradient
6. SPSA
7. Summary & Exercises

Motivation: Why Derivatives?

- Optimization: find design point that **minimises** (or maximises) an objective function.
- The *derivative* tells us **how the function changes** as we vary an input.
- Knowing the derivative allows us to predict which direction improves the objective.
- In one dimension: **derivative** $f'(x)$.
In multiple dimensions: **gradient** $\nabla f(\mathbf{x})$.



The derivative equals the slope of the tangent line.

Scalar Derivative: Definition

Definition (2.2)

The **derivative** $f'(x)$ of a function $f : \mathbb{R} \rightarrow \mathbb{R}$ at x is the rate at which f changes:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} = \frac{\Delta f(x)}{\Delta x}$$

Interpretations

- **Geometric:** slope of the tangent line.
- **Linear approximation:**
 $f(x + \Delta x) \approx f(x) + f'(x) \Delta x$
- **Leibniz notation:** $f'(x) \equiv \frac{df(x)}{dx}$

Three equivalent limit forms (eq. 2.4):

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \quad (\text{forward})$$

$$= \lim_{h \rightarrow 0} \frac{f(x+h/2) - f(x-h/2)}{h} \quad (\text{central})$$

$$= \lim_{h \rightarrow 0} \frac{f(x) - f(x-h)}{h} \quad (\text{backward})$$

Symbolic Differentiation (Example 2.1)

Chain rule / product rule

Given $f(x) = x^2 + x/2 - \sin(x)/x$, symbolic differentiation yields the exact closed-form derivative.

Using SymPy (Python equivalent of Julia's SymEngine):

```
from sympy import symbols, diff, sin
x = symbols('x')
f = x**2 + x/2 - sin(x)/x
df = diff(f, x)
# Result:
# 2*x + 1/2 + sin(x)/x**2 - cos(x)/x
```

Result:

$$f'(x) = 2x + \frac{1}{2} + \frac{\sin x}{x^2} - \frac{\cos x}{x}$$

- Exact (no numerical error).
- May produce *expression swell* for complex functions.
- Outside scope of this chapter; provided by sympy.

Gradient

Definition — Gradient (eq. 2.5)

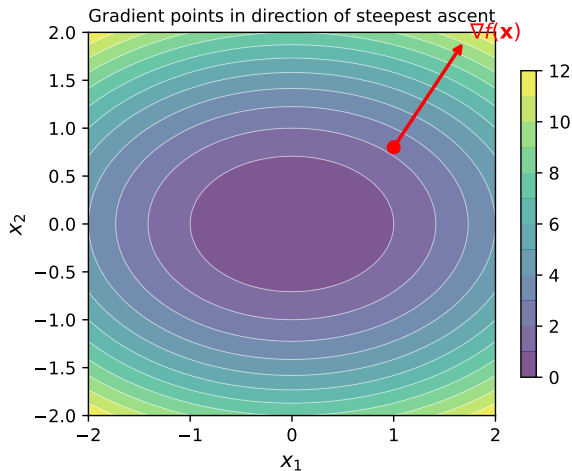
For $f : \mathbb{R}^n \rightarrow \mathbb{R}$, the **gradient** is the vector of partial derivatives:

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix} \in \mathbb{R}^n$$

x_i — i -th component of $\mathbf{x}$; $\partial f / \partial x_i$ — partial derivative w.r.t. x_i .

- The gradient points in the direction of **steepest ascent**.
- A **hyperplane** (tangent) can be used to approximate f near $\mathbf{x}$: $f(\mathbf{x} + \Delta \mathbf{x}) \approx f(\mathbf{x}) + \nabla f(\mathbf{x})^\top \Delta \mathbf{x}$.

Gradient: Geometric Intuition



Key observations:

- $\nabla f(\mathbf{x})$ is *perpendicular* to level curves.
- Moving *opposite* the gradient is the steepest descent direction.
- For $f(\mathbf{x}) = x_1^2 + 2x_2^2$:

$$\nabla f(\mathbf{x}) = \begin{bmatrix} 2x_1 \\ 4x_2 \end{bmatrix}$$

- Gradient-based optimisers use $-\nabla f(\mathbf{x})$ to update $\mathbf{x}$.

Jacobian Matrix

Definition — Jacobian

For $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$, the **Jacobian** is the $m \times n$ matrix of partial derivatives:

$$\mathbf{J}_{ij} = \frac{\partial f_i}{\partial x_j}, \quad \mathbf{J} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

- For scalar f ($m = 1$): Jacobian $= \nabla f(\mathbf{x})^\top$ (row vector).
- Chain rule for vector functions: $\frac{d\mathbf{f}(\mathbf{g}(\mathbf{x}))}{d\mathbf{x}} = \mathbf{J}_f \cdot \mathbf{J}_g$.

Hessian Matrix

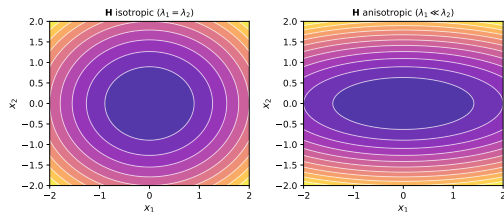
Definition — Hessian (eq. 2.8)

For $f : \mathbb{R}^n \rightarrow \mathbb{R}$, the **Hessian** is the symmetric $n \times n$ matrix of second-order partial derivatives:

$$\mathbf{H}_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$$

$$\mathbf{H} = \begin{bmatrix} f_{x_1 x_1} & \cdots & f_{x_1 x_n} \\ \vdots & \ddots & \vdots \\ f_{x_n x_1} & \cdots & f_{x_n x_n} \end{bmatrix}$$

- Captures **curvature** of f near $\mathbf{x}$.
- Used in second-order optimisers (Newton's method).
- Symmetric when f is twice continuously differentiable.



Top: isotropic curvature ($\lambda_1 = \lambda_2$). Bottom: anisotropic — elongated contours.

Directional Derivative

Definition (eq. 2.10)

The **directional derivative** of f at $\mathbf{x}$ in direction $\mathbf{s} \in \mathbb{R}^n$ is:

$$\nabla_{\mathbf{s}} f(\mathbf{x}) = \lim_{h \rightarrow 0} \frac{f(\mathbf{x} + h\mathbf{s}) - f(\mathbf{x})}{h} = \nabla f(\mathbf{x})^{\top} \mathbf{s}$$

Interpretation:

- Rate of change of f along direction $\mathbf{s}$.
- If $\|\mathbf{s}\| = 1$, it is the *slope* in direction $\mathbf{s}$.
- Special case: $\mathbf{s} = \mathbf{e}_i$ gives $\partial f / \partial x_i$.

Numerical example:

$$f(x_1, x_2) = x_1^2 + x_2^2, \mathbf{x} = (1, 1)^{\top}, \\ \mathbf{s} = (1/\sqrt{2}, 1/\sqrt{2})^{\top}:$$

$$\nabla f = (2, 2)^{\top}$$

$$\nabla_{\mathbf{s}} f = (2, 2) \cdot \frac{1}{\sqrt{2}}(1, 1)^{\top} = 2\sqrt{2}$$

Finite Difference Methods: Overview

- **Approximate** derivatives by evaluating f at nearby points.
- Three methods based on different limit interpretations of eq. (2.4):

Finite Difference Approximations

Method	Formula	Error
Forward difference	$\frac{f(x+h) - f(x)}{h}$	$O(h)$
Central difference	$\frac{f(x+h/2) - f(x-h/2)}{h}$	$O(h^2)$
Backward difference	$\frac{f(x) - f(x-h)}{h}$	$O(h)$

Central difference is more accurate (quadratic vs. linear error) but requires two function evaluations.

Forward Difference: Taylor Analysis

Taylor Expansion (eq. 2.11–2.13)

Expanding $f(x + h)$ around x :

$$f(x + h) = f(x) + f'(x)h + \frac{f''(x)}{2!}h^2 + \frac{f'''(x)}{3!}h^3 + \dots$$

Rearranging:

$$f'(x) = \frac{f(x + h) - f(x)}{h} - \frac{f''(x)}{2}h - \dots \approx \frac{f(x + h) - f(x)}{h} + O(h)$$

Observations:

- Error term is $O(h)$ — linear in step size.
- Smaller $h \Rightarrow$ better approximation *in principle*.
- **But:** too small h causes *subtractive cancellation* (floating-point loss).

Practical tip: Choose $h \approx \sqrt{\epsilon_{\text{mach}}} \approx 10^{-8}$ for forward difference ($\epsilon_{\text{mach}} \approx 2.2 \times 10^{-16}$ in float64).

Central Difference: Taylor Analysis

Taylor expansions (eq. 2.15–2.18)

$$f(x + h/2) = f(x) + f'(x)\frac{h}{2} + \frac{f''(x)}{2!}\left(\frac{h}{2}\right)^2 + \frac{f'''(x)}{3!}\left(\frac{h}{2}\right)^3 + \dots$$

$$f(x - h/2) = f(x) - f'(x)\frac{h}{2} + \frac{f''(x)}{2!}\left(\frac{h}{2}\right)^2 - \frac{f'''(x)}{3!}\left(\frac{h}{2}\right)^3 + \dots$$

Subtracting and rearranging:

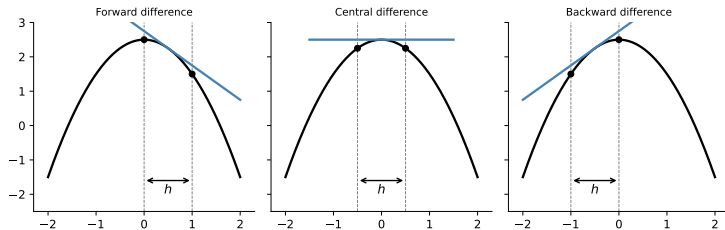
$$f'(x) \approx \frac{f(x + h/2) - f(x - h/2)}{h} + O(h^2)$$

- Error is $O(h^2)$ — **quadratic** accuracy.
- Even-order terms cancel $\Rightarrow$ faster convergence.
- Optimal step: $h \approx \epsilon_{\text{mach}}^{1/3} \approx 10^{-5}$.

Finite Difference: Python Implementation

```
1 import numpy as np
2
3 def diff_forward(f, x, h=1e-8):
4     """Forward difference:  $O(h)$  error."""
5     return (f(x + h) - f(x)) / h
6
7 def diff_central(f, x, h=1e-5):
8     """Central difference:  $O(h^2)$  error."""
9     return (f(x + h/2) - f(x - h/2)) / h
10
11 def diff_backward(f, x, h=1e-8):
12     """Backward difference:  $O(h)$  error."""
13     return (f(x) - f(x - h)) / h
14
15 # Gradient for multivariate  $f: \mathbb{R}^n \rightarrow \mathbb{R}$  (forward difference)
16 def gradient_fd(f, x, h=1e-5):
17     """Numerical gradient using central differences."""
18     n = len(x)
19     g = np.zeros(n)
20     for i in range(n):
21         xp, xm = x.copy(), x.copy()
22         xp[i] += h/2; xm[i] -= h/2
23         g[i] = (f(xp) - f(xm)) / h
24     return g
25
26 # Running example:  $f(x) = \sin(x^2)$ ,  $x = \pi/2$ 
27 f = lambda x: np.sin(x**2)
```

Finite Difference: Visualisation



The tangent line is approximated by joining two nearby points. Central difference uses a symmetric bracket around x .

Complex-Step Method

Key Idea (eq. 2.19–2.22)

Evaluate f at a **complex** point $x + ih$ where $i = \sqrt{-1}$:

$$f(x + ih) = f(x) + ihf'(x) - \frac{h^2 f''(x)}{2!} - \frac{ih^3 f'''(x)}{3!} + \dots$$

Taking the imaginary part:

$$\text{Im}(f(x + ih)) = hf'(x) - \frac{h^3 f'''(x)}{3!} + \dots$$

$$f'(x) \approx \frac{\text{Im}(f(x + ih))}{h} + O(h^2)$$

- **No subtractive cancellation** — uses imaginary part, not difference.
- Optimal h : choose h extremely small, e.g., $h = 10^{-20}$ is fine.
- Real part gives $f(x)$ to within $O(h^2)$: $\text{Re}(f(x + ih)) \approx f(x) - \frac{h^2 f''(x)}{2} + \dots$
- **Requirement:** f must support complex inputs (analytic f).

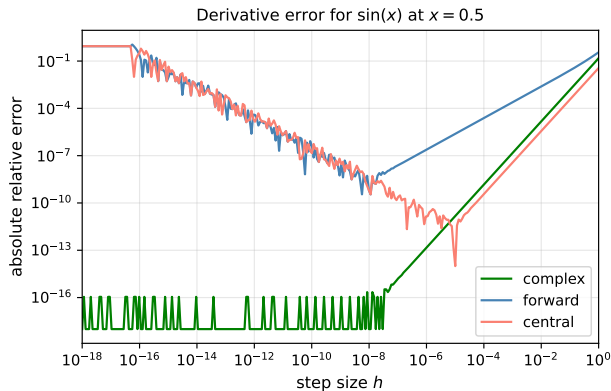
Complex-Step Method: Python Code + Example

```
1 import numpy as np
2
3 def diff_complex(f, x, h=1e-20):
4     """Complex-step derivative:  $\text{Im}(f(x + i*h)) / h$ """
5     return np.imag(f(x + h*1j)) / h
6
7 # Example 2.4:  $f(x) = \sin(x^2)$ 
8 f = lambda x: np.sin(x**2)
9 x0 = np.pi / 2
10
11 # function value and derivative using a single evaluation
12 v = f(x0 + 1e-20j)
13 fx = np.real(v) #  $f(x_0)$ 
14 dfdx = np.imag(v) / 1e-20 #  $f'(x_0)$ 
15
16 print(f"f(pi/2) = {fx:.8f}") # approx 0.62426
17 print(f"f'(pi/2) = {dfdx:.8f}") # approx -2.45425
18 # True:  $\pi * \cos(\pi^2/4)$ 
19 true_d = np.pi * np.cos(np.pi**2 / 4)
20 print(f"True = {true_d:.8f}")
```

Listing 2: Complex-step derivative (Algorithm 2.2 equivalent)

Key output: Both $f(x)$ and $f'(x)$ from a *single* function evaluation.

Error Comparison: Forward vs. Central vs. Complex-Step



Observations:

- **Forward difference:** linear error ($O(h)$) — slope -1 on left.
- **Central difference:** quadratic error ($O(h^2)$) — slope -2 .
- **Complex-step:** stays near machine precision for all small h !
- Forward/central exhibit *cancellation errors* as $h \rightarrow 0$.
- Complex-step avoids cancellation entirely.

Test function: $\sin(x)$ at $x = 0.5$; true derivative $\cos(0.5)$.

Automatic Differentiation: Motivation

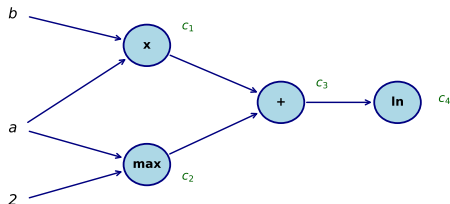
- Numerical finite differences are simple but have accuracy/cost tradeoffs.
- Symbolic differentiation can suffer from *expression swell*.
- **Automatic differentiation (AD)** computes *exact* derivatives of a function specified as computer code.
- Based on the **chain rule**:

$$\frac{d}{dx} f(g(x)) = \frac{df}{dg} \cdot \frac{dg}{dx}$$

- A program consists of elementary operations: $+$, $-$, $\times$, $\div$, $\sin$, $\exp$, $\ln$, $\dots$ — each has a known derivative.
- The **computational graph** encodes data flow.

Computational Graph

Computational graph for $f(a, b) = \ln(ab + \max(a, 2))$



Computational graph for $f(a, b) = \ln(ab + \max(a, 2))$.

Structure:

- **Leaf nodes:** inputs (a , b) and constants (2).
- **Internal nodes:** operations ($\times$, $\max$, $+$, $\ln$).
- **Terminal node:** output (c_4).

Intermediate values:

$$c_1 = a \times b$$

$$c_2 = \max(a, 2)$$

$$c_3 = c_1 + c_2$$

$$c_4 = \ln(c_3) = f(a, b)$$

Two traversal strategies: **forward** and **reverse** accumulation.

Forward Accumulation I

Forward Mode: Chain Rule (eq. 2.31)

Traverse the graph from **inputs to outputs**, propagating (value, $\dot{c}$) pairs where $\dot{c}_i = \partial c_i / \partial x$:

$$\frac{df}{dx} = \frac{df}{dc_4} \cdot \frac{dc_4}{dc_3} \cdot \left(\frac{dc_3}{dc_2} \cdot \frac{dc_2}{dx} + \frac{dc_3}{dc_1} \cdot \frac{dc_1}{dx} \right)$$

Worked example (eq. 2.26–2.30): $f(a, b) = \ln(ab + \max(a, 2))$ at $a = 3$, $b = 2$, computing $\partial f / \partial a$:

Node	Value	Derivative $\dot{c}$
a	3	$\dot{a} = 1$
b	2	$\dot{b} = 0$
$c_1 = ab$	6	$\dot{c}_1 = b\dot{a} + a\dot{b} = 2$
$c_2 = \max(a, 2)$	3	$\dot{c}_2 = \dot{a} = 1$ (since $2 < a$)
$c_3 = c_1 + c_2$	9	$\dot{c}_3 = \dot{c}_1 + \dot{c}_2 = 3$
$c_4 = \ln c_3$	$\ln 9$	$\dot{c}_4 = \dot{c}_3 / c_3 = 1/3$

Result:

$$f(3, 2) = \ln 9, \quad \frac{\partial f}{\partial a} = \frac{1}{3}$$

- **One forward pass** computes one partial derivative.
- For n inputs: requires n passes.
- Efficient when **few inputs** (e.g. $n \ll m$).

Forward Accumulation II

Dual numbers: A clean way to implement forward accumulation.

Dual Number Arithmetic (eq. 2.32–2.33)

Introduce abstract quantity ϵ with $\epsilon^2 = 0$. A dual number $a + b\epsilon$ encodes both value (a) and derivative (b):

$$(a + b\epsilon) + (c + d\epsilon) = (a + c) + (b + d)\epsilon$$

$$(a + b\epsilon) \times (c + d\epsilon) = (ac) + (ad + bc)\epsilon$$

- Set initial dual number: $x = x_0 + 1 \cdot \epsilon$ to compute $f'(x_0)$.
- Propagate through all operations naturally.

Forward AD: Python with Dual Numbers

```
1 import numpy as np
2
3 class Dual:
4     """Dual number: v + d*epsilon where epsilon^2 = 0."""
5     def __init__(self, v, d=0.0):
6         self.v, self.d = float(v), float(d)
7
8     def __add__(self, other):
9         if isinstance(other, Dual):
10             return Dual(self.v + other.v, self.d + other.d)
11         return Dual(self.v + other, self.d)
12     __radd__ = __add__
13
14     def __mul__(self, other):
15         if isinstance(other, Dual):
16             return Dual(self.v * other.v,
17                         self.v * other.d + self.d * other.v)
18         return Dual(self.v * other, self.d * other)
19     __rmul__ = __mul__
20
21     def log(self):
22         return Dual(np.log(self.v), self.d / self.v)
23
24 # Example 2.5: f(a,b) = log(a*b + max(a,2)) at a=3, b=2
25 def f(a, b):
26     c1 = a * b
27     c2 = Dual(max(a.v, 2), a.d if a.v > 2 else 0.0)
```

Reverse Accumulation (Backpropagation) I

Reverse Mode: Chain Rule

Traverse the graph from **outputs to inputs**, propagating **adjoints** (sensitivities) $\bar{c}_i = \partial f / \partial c_i$:

$$\bar{c}_i = \sum_{j: j \text{ depends on } i} \bar{c}_j \cdot \frac{\partial c_j}{\partial c_i}$$

Procedure:

- 1 **Forward pass:** compute and store all intermediate values c_i (the *tape*).
- 2 **Backward pass:** propagate adjoints from output to inputs using stored values.

Complexity:

- One backward pass computes **all n partial derivatives** simultaneously.
- Cost $\approx 3\times$ forward pass.
- Ideal when **few outputs, many inputs** (common in ML!).

Reverse Accumulation (Backpropagation) II

Forward vs. Reverse — when to use which:

	Forward mode	Reverse mode
Passes needed	n (one per input)	m (one per output)
Best for	$n \ll m$ (few inputs)	$m \ll n$ (few outputs)
Memory	$O(1)$	$O(\text{graph size})$
ML / deep learning	less common	backpropagation

In deep learning: $f : \mathbb{R}^n \rightarrow \mathbb{R}$ (scalar loss), n huge $\Rightarrow$ reverse mode (backprop) computes full gradient in one pass.

Reverse AD with JAX (Python)

```
1  # pip install jax
2  import jax
3  import jax.numpy as jnp
4
5  # Define function
6  def f(x):
7      return jnp.log(x[0]*x[1] + jnp.maximum(x[0], 2.0))
8
9  # Reverse mode: full gradient in one pass
10 x0 = jnp.array([3.0, 2.0])
11 grad_f = jax.grad(f)           # returns gradient function
12 g = grad_f(x0)                 # [df/da, df/db]
13 print(f"df/da = {g[0]:.4f}")   # 0.3333
14 print(f"df/db = {g[1]:.4f}")   # 0.3333
15
16 # Jacobian-vector product (forward mode)
17 primals, tangents = jax.jvp(f, (x0,), (jnp.array([1.0, 0.0]),))
18 print(f"JVP (df/da) = {tangents:.4f}") # 0.3333
19
20 # Hessian
21 hess = jax.hessian(f)(x0)
22 print("Hessian:\n", hess)
```

Listing 4: Automatic differentiation with JAX (forward + reverse mode)

Regression Gradient: Motivation

- Sometimes f is a **black-box**: no closed form, no AD support.
- Idea: estimate the gradient from *multiple function evaluations* using linear regression.
- Take m perturbation vectors $\Delta \mathbf{x}^{(i)}$ and evaluate $f(\mathbf{x} + \Delta \mathbf{x}^{(i)})$.
- Fit the best linear model: $f(\mathbf{x} + \Delta \mathbf{x}) \approx f(\mathbf{x}) + \mathbf{g}^\top \Delta \mathbf{x}$

Optimisation problem (eq. 2.42)

$$\min_{\mathbf{g}} \sum_{i=1}^m \left(f(\mathbf{x} + \Delta \mathbf{x}^{(i)}) - \left(f(\mathbf{x}) + \mathbf{g}^\top \Delta \mathbf{x}^{(i)} \right) \right)^2$$

Regression Gradient: Solution (Algorithm 2.3)

Matrix formulation (eq. 2.43–2.46)

Define:

$$\Delta \mathbf{X} = \begin{bmatrix} (\Delta \mathbf{x}^{(1)})^\top \\ \vdots \\ (\Delta \mathbf{x}^{(m)})^\top \end{bmatrix}, \quad \Delta \mathbf{f} = \begin{bmatrix} f(\mathbf{x} + \Delta \mathbf{x}^{(1)}) - f(\mathbf{x}) \\ \vdots \\ f(\mathbf{x} + \Delta \mathbf{x}^{(m)}) - f(\mathbf{x}) \end{bmatrix}$$

Problem becomes: $\min_{\mathbf{g}} \|\Delta \mathbf{X} \mathbf{g} - \Delta \mathbf{f}\|^2$

Closed-form solution via **pseudoinverse**:

$$\mathbf{g} = \Delta \mathbf{X}^+ \Delta \mathbf{f}$$

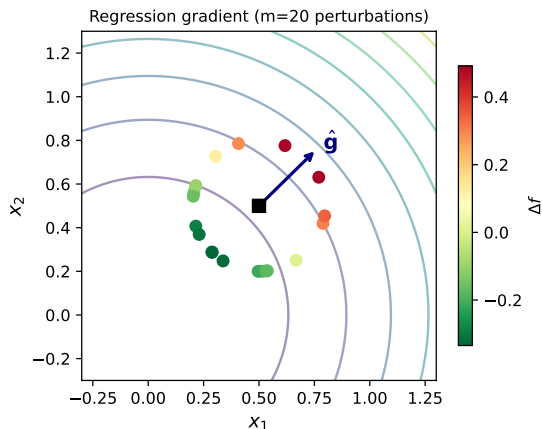
where $\Delta \mathbf{X}^+$ is the Moore–Penrose pseudoinverse.

Perturbations: draw $\Delta \mathbf{x}^{(i)}$ uniformly from a hypersphere of radius δ .

Regression Gradient: Python Implementation

```
1 import numpy as np
2
3 def regression_gradient(f, x, m=20, delta=0.1):
4     """
5     Estimate gradient of f at x using m perturbations
6     drawn uniformly from a hypersphere of radius delta.
7     """
8     n = len(x)
9     fx = f(x)
10    # Generate m random unit directions, scaled by delta
11    dX = np.random.randn(m, n)
12    dX /= np.linalg.norm(dX, axis=1, keepdims=True) # unit sphere
13    dX *= delta # scale to radius delta
14    # Evaluate function differences
15    df = np.array([f(x + dx) - fx for dx in dX])
16    # Least-squares solution:  $g = dX^+ @ df$ 
17    g, *_ = np.linalg.lstsq(dX, df, rcond=None)
18    return g
19
20 # Test on  $f(x) = x_1^2 + x_2^2$ , true gradient at (1,1): [2, 2]
21 f = lambda x: x[0]**2 + x[1]**2
22 x0 = np.array([1.0, 1.0])
23 np.random.seed(42)
24 g_est = regression_gradient(f, x0, m=50, delta=0.01)
25 print(f"Estimated gradient: {g_est}") # ~[2.0, 2.0]
26 print(f"True gradient: [2.0, 2.0]")
```

Regression Gradient: Illustration



Interpretation:

- Red/green points: perturbations coloured by Δf .
- Navy arrow: estimated gradient direction.
- More perturbations $\Rightarrow$ better estimate.
- Works for **non-differentiable** or black-box f .
- Complexity: $O(mn)$ for m evaluations in n dimensions.

Limitations:

- Requires $m \geq n$ evaluations.
- Noisy for large n .
- SPSA more efficient for very large n .

Simultaneous Perturbation Stochastic Gradient Approximation

Key Idea (SPSA)

Estimate the gradient using only **2 function evaluations**, regardless of dimension n , by perturbing *all* coordinates simultaneously with a random ± 1 direction vector $\mathbf{z}$.

SPSA Gradient Estimate (eq. 2.48)

For a random vector $\mathbf{z} \in \{-1, +1\}^n$:

$$\nabla_{\mathbf{z}} f(\mathbf{x}) \approx \frac{f(\mathbf{x} + \delta \mathbf{z}) - f(\mathbf{x} - \delta \mathbf{z})}{2\delta}$$

The component-wise gradient estimate:

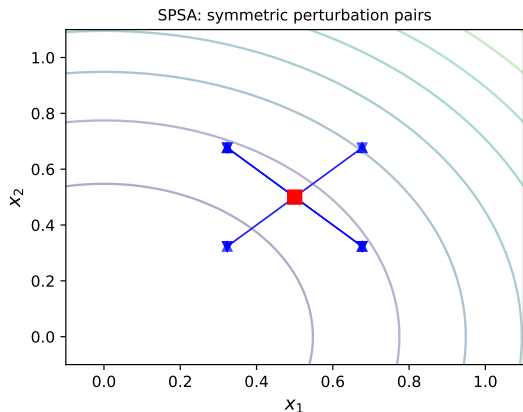
$$\hat{g}_i \approx \frac{f(\mathbf{x} + \delta \mathbf{z}) - f(\mathbf{x} - \delta \mathbf{z})}{2\delta} \cdot z_i$$

- Average over m samples: $\hat{\mathbf{g}} = \frac{1}{m} \sum_{k=1}^m \hat{\mathbf{g}}^{(k)}$.
- Unbiased: $\mathbb{E}[\hat{g}_i] = \partial f / \partial x_i$.

SPSA: Python Implementation

```
1 import numpy as np
2
3 def spsa_gradient(f, x, delta=0.1, m=10):
4     """
5     SPSA gradient estimate using m pairs of perturbations.
6     Each pair requires only 2 function evaluations.
7     Total cost: 2*m function evaluations (vs n+1 for finite diff).
8     """
9     n = len(x)
10    g = np.zeros(n)
11    for _ in range(m):
12        # Random +/-1 perturbation direction
13        z = np.random.choice([-1.0, 1.0], size=n)
14        fp = f(x + delta * z)
15        fm = f(x - delta * z)
16        g += ((fp - fm) / (2 * delta)) * z
17    return g / m
18
19 # Example:  $f(x,y) = x^2 + y^2 + 0.5xy$ 
20 f = lambda x: x[0]**2 + x[1]**2 + 0.5*x[0]*x[1]
21 x0 = np.array([1.0, 2.0])
22 # True gradient:  $[2*x_0 + 0.5*y_0, 2*y_0 + 0.5*x_0] = [3.0, 4.5]$ 
23 np.random.seed(0)
24 g_spsa = spsa_gradient(f, x0, delta=0.01, m=500)
25 print(f"SPSA estimate: {g_spsa}")
26 print(f"True gradient: [3.0, 4.5]")
```


SPSA: Illustration and Comparison



SPSA vs. Regression Gradient:

	Reg. Grad.	SPSA
Evaluations	$m + 1$	$2m$
Per estimate	$m \geq n$	$m = 1$ works
Accuracy	Better	Noisier
Large n	Expensive	Scalable

- SPSA is central to **black-box optimisation**.
- Widely used in RL, neural architecture search.
- Noisy but *unbiased* on average.

Chapter Summary

Core Concepts

- **Derivative** $f'(x)$: instantaneous rate of change.
- **Gradient** $\nabla f(\mathbf{x})$: steepest ascent direction.
- **Jacobian**: first-order multivariate derivatives.
- **Hessian**: second-order; captures curvature.
- **Directional derivative**: rate along a vector.

Differentiation Methods

- **Symbolic**: exact but may have expression swell.
- **Forward diff**: $O(h)$ error, 2 evaluations.
- **Central diff**: $O(h^2)$ error, 2 evaluations.
- **Complex-step**: $O(h^2)$, no cancellation.
- **Forward AD**: exact, n passes for n inputs.
- **Reverse AD**: exact, 1 backward pass, all derivatives.
- **Regression gradient**: black-box, $m + 1$ evals.
- **SPSA**: black-box, $2m$ evals, scalable.

Summary Bullet Points (Section 2.7) I

- Derivatives are useful in optimisation because they provide information about how to change a given point to **improve** the objective function.
- For multivariate functions, the **gradient**, **Hessian**, and **directional derivative** are the key derivative concepts for directing the search for an optimum.
- One approach to numerical differentiation includes **finite difference approximations**: forward, central, and backward methods.
- The **complex-step method** can eliminate subtractive cancellation error when taking small steps, resulting in high-quality gradient estimates.
- **Automatic differentiation**: forward and reverse accumulation on computational graphs give exact numerical derivatives at code level.
- The **regression gradient** uses linear regression to find the best estimate of the gradient with respect to a set of sampled perturbations and their function evaluations.
- **SPSA** allows estimation of gradients for very large objective functions using only 2 function evaluations per iteration.

Selected Exercises I

Exercise 2.1. Approximate the Hessian from the gradient using forward differences.

Solution

$$H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j} \approx \frac{[\nabla f(\mathbf{x} + h\mathbf{e}_j)]_i - [\nabla f(\mathbf{x})]_i}{h}$$

Repeat for each column $j = 1, \dots, n$.

Exercise 2.2. What is a drawback of central differences?

Solution

Central difference requires **two function evaluations** per dimension (versus one for forward/backward), doubling cost.

Exercise 2.3. Find the dominant term in $f'(x) = \frac{1}{x} + e^x - \frac{1}{x^2}$ for x close to zero.

Selected Exercises II

Solution

For $x < 1$: $\frac{1}{x} > 1$, so $\frac{1}{x^2} > \frac{1}{x} \gg e^x$. The term $-\frac{1}{x^2}$ dominates.

Selected Exercises III

Exercise 2.4. If $f(3 + ih) = 2 + 4ih$, what is $f'(3)$?

Solution

Complex-step: $f'(3) \approx \text{Im}(f(3 + ih))/h = 4h/h = 4$.

Exercise 2.6. Combine forward and backward differences to estimate $f''(x)$.

Solution

$$f''(x) \approx \frac{f'(x + h/2) - f'(x - h/2)}{h} \approx \frac{f(x + h) - 2f(x) + f(x - h)}{h^2}$$

Three function evaluations; $O(h^2)$ accuracy.

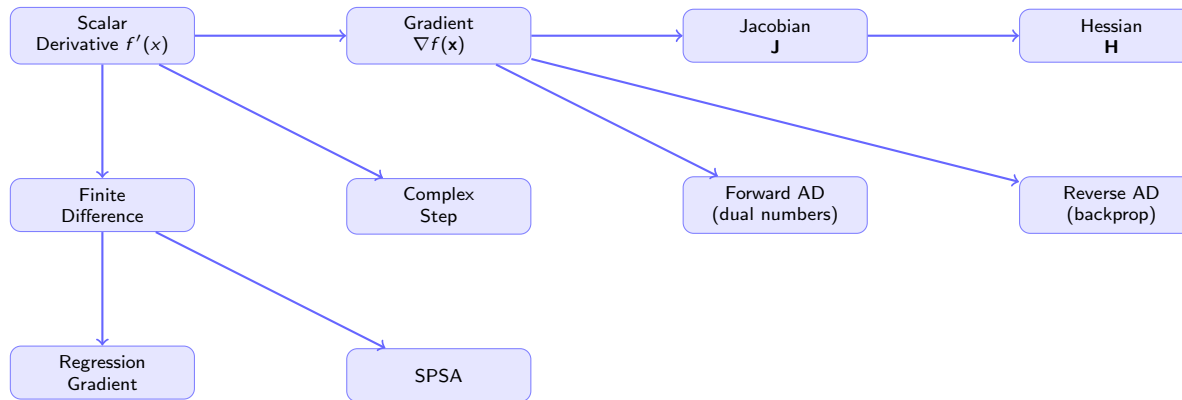
Exercise 2.5. Computational graph for $f(x, y) = \sin(x + y^2)$; forward accumulation at $(x, y) = (1, 1)$ for $\partial f / \partial y$:

Selected Exercises IV

$$y \xrightarrow{(\cdot)^2} c_1 = 1; \quad x \rightarrow c_2 = x = 1; \quad c_1 + c_2 \rightarrow c_3 = 2; \quad \sin(c_3) \rightarrow c_4 = \sin 2$$
$$\dot{y} = 1; \quad \dot{c}_1 = 2y \cdot \dot{y} = 2; \quad \dot{c}_2 = 0; \quad \dot{c}_3 = 2; \quad \dot{c}_4 = 2 \cos 2$$

$$\Rightarrow \partial f / \partial y = 2 \cos 2 \approx -0.832.$$

Chapter 2: Concept Map



Method Selection Guide

When to use which differentiation method

Scenario	Method	Reason
Analytic f , closed form	Symbolic	Exact; fast evaluation
Code-defined f , few inputs	Forward AD	One pass per input
Code-defined f , scalar output	Reverse AD	One backward pass
Black-box, smooth f	Complex-step	High accuracy, no cancellation
Black-box, moderate n	Regression gradient	Robust, $O(n)$ evals
Black-box, very large n	SPSA	Only 2 evals/iter

References

-  M. J. Kochenderfer and T. A. Wheeler, *Algorithms for Optimization*, 2nd ed. MIT Press, 2026.
-  J. R. R. A. Martins, P. Sturdza, and J. J. Alonso, “The Complex-Step Derivative Approximation,” *ACM Trans. Math. Softw.*, vol. 29, no. 3, pp. 245–262, 2003.
-  A. G. Baydin et al., “Automatic Differentiation in Machine Learning: a Survey,” *JMLR*, vol. 18, no. 153, 2018.
-  J. C. Spall, “Multivariate Stochastic Approximation Using a Simultaneous Perturbation Gradient Approximation,” *IEEE Trans. Autom. Control*, vol. 37, no. 3, pp. 332–341, 1992.